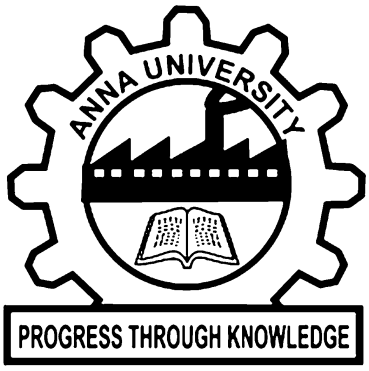
BUILD A MEDICAL CHATBOT USING RAG ARCHITECTURE WITH COHERE LLM

A SUMMER INTERNSHIP REPORT

Submitted by

J. MUHAMMED AARIF

Register Number: 810023205044



DEPARTMENT OF INFORMATION TECHNOLOGY

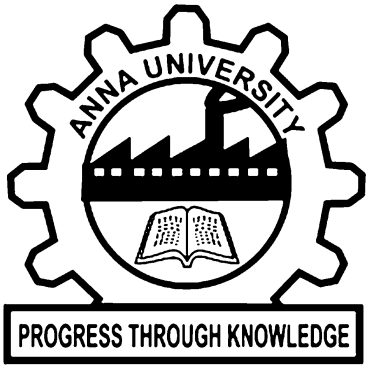
UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS,

ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS,

ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify that the summer internship project report **"BUILD A MEDICAL CHATBOT USING RAG ARCHITECTURE WITH COHERE LLM"** is the bonafide work of 810023205044, a student of B.Tech. Information Technology, Semester VII, University College of Engineering, BIT Campus, Anna University, Tiruchirappalli, undertaken at **ASTONISH INFOTECH PVT. LTD.**, Tiruchirappalli, from 1 June 2026 to 30 June 2026, during 2026.

FACULTY INCHARGE **HEAD OF THE DEPARTMENT**

Submitted for the practical examination held on _____

ABSTRACT

End-to-End Medical Care Generative AI Chatbot, named CareBot in the supplied repository, is a web-based question-answering application designed to respond to medical and health-related questions using a medical PDF knowledge source. The project combines a Flask web interface with a Retrieval-Augmented Generation (RAG) pipeline. The medical document is loaded and split into smaller text chunks, which can be embedded with a Hugging Face sentence-transformer model and indexed in Pinecone. When a user asks a question, the application retrieves the most similar text passages and supplies them as context to a Cohere language model to generate a concise response.

The repository includes an application entry point, helper functions for PDF loading and chunking, a medical-only prompt, a vector-index storage script, HTML and CSS interface files, screenshots, and a medical-book PDF. The RAG approach is intended to ground generated answers in retrieved content rather than relying only on a model's general knowledge. The prompt also instructs the assistant to decline questions outside the medical domain and to acknowledge when it does not know an answer.

This report explains the project motivation, objectives, related concepts, system requirements, architecture, data preparation, embedding and retrieval process, response generation, interface, implementation, testing approach, limitations, privacy and safety considerations, and possible future improvements. Because the application concerns health information, its output must be treated as informational only. Clinical validation, secure deployment, monitoring, and professional review are necessary before any real-world healthcare use.

Keywords: Generative AI, medical chatbot, Retrieval-Augmented Generation, LangChain, Flask, Hugging Face embeddings, Pinecone, Cohere, healthcare information.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Abstract	3
2	List of Figures	5
3	List of Tables	5
4	Chapter 1 – Introduction	6
5	Chapter 2 – Problem Statement and Objectives	8
6	Chapter 3 – Background and Related Concepts	11
7	Chapter 4 – Existing and Proposed System	12
8	Chapter 5 – Requirement Analysis	13
9	Chapter 6 – System Architecture	14
10	Chapter 7 – Dataset and Knowledge Preparation	15
11	Chapter 8 – Embeddings and Vector Search	16
12	Chapter 9 – Retrieval-Augmented Generation	17
13	Chapter 10 – Prompt Design and Response Policy	18
14	Chapter 11 – Technology Stack	19
15	Chapter 12 – Backend Implementation	20
16	Chapter 13 – Frontend and User Interaction	21
17	Chapter 14 – End-to-End Workflow	22
18	Chapter 15 – Setup and Deployment	23
19	Chapter 16 – Testing Strategy	24
20	Chapter 17 – Test Cases and Observations	28
21	Chapter 18 – Conclusion	29
28	References	30

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1.1	CareBot RAG architecture	9
Figure 6.1	End-to-end system components	14
Figure 7.1	Medical knowledge ingestion pipeline	15
Figure 9.1	RAG answer generation flow	17
Figure 13.1	Chatbot interface screenshot 1	21
Figure 13.2	Chatbot interface screenshot 2	21
Figure 13.3	Chatbot interface screenshot 3	21
Figure 18.1	Safe response flow	26

LIST OF TABLES

Table No.	Table Name	Page No.
Table 4.1	Existing and proposed system comparison	12
Table 5.1	Functional requirements	13
Table 5.2	Non-functional requirements	13
Table 11.1	Technology stack	19
Table 17.1	Sample test cases	25
Table 19.1	Qualitative evaluation criteria	27
Table 20.1	Known limitations and impact	28
Table D.1	Glossary of terms	37

CHAPTER 1 – INTRODUCTION

1.1 Overview

Healthcare information is widely available online, but users may find it difficult to locate a short, relevant explanation within long documents. A conversational interface can make document-based information easier to explore. CareBot is a medical question-answering chatbot that accepts a user question and returns a response generated with the help of retrieved passages from a medical PDF.

1.2 Project Motivation

Large language models can produce fluent responses, but they may also generate unsupported statements. Retrieval-Augmented Generation adds a search step so that relevant source text can be provided to the model. The supplied project applies this pattern to a medical knowledge source and keeps the prompt focused on health-related questions.

1.3 Scope

The documented scope includes a Flask application, a browser-based chat interface, PDF loading and text splitting, sentence-transformer embeddings, a Pinecone vector index, and a Cohere language model. The project is a prototype for educational exploration and does not establish clinical accuracy or medical-device readiness. The second part involves generating embeddings and storing them in a vector database. The project uses the Hugging Face sentence-transformer model sentence-transformers/all-MiniLM-L6-v2 to convert text into numerical representations. Pinecone is used to store these vectors and support similarity-based retrieval. This process allows the system to search for passages that are semantically related to a question.

The third part of the project covers the Retrieval-Augmented Generation pipeline. When a user enters a question, the system searches the vector database for relevant document chunks. The retrieved information is combined with the question and passed to the Cohere language model

through the configured LangChain workflow. The model generates a response using the supplied context and prompt instructions.

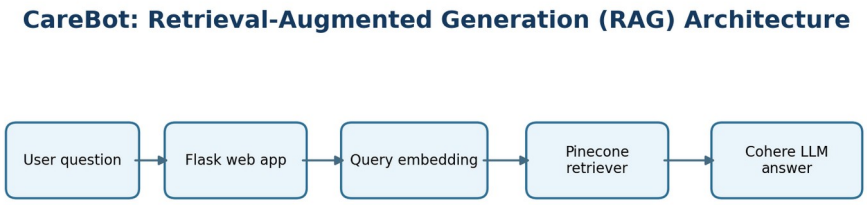


Figure 1.1: High-level CareBot architecture

1.4 Need for the System

The project demonstrates a practical way to make a fixed collection of medical information easier to explore. Instead of asking the user to locate a page and interpret a long passage alone, the system accepts a natural-language question and prepares a short response using retrieved document text. The project also demonstrates that successful AI development involves more than generating fluent text. This can improve convenience for learning and initial information discovery, provided that the user understands the limitations of generated answers.

1.5 Significance

The academic value of the project lies in integrating several modern application components into one workflow. It brings together document processing, semantic search, prompt design, language generation, and a browser interface. The same pattern can be adapted to other carefully reviewed knowledge collections, although medical applications require stronger privacy, quality assurance, and safety controls than ordinary demonstration chatbots.

CHAPTER 2 – PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Users may need a simple way to ask questions about a medical reference document without manually searching every page. A basic keyword search may miss semantically similar wording, while a general-purpose chatbot may answer without grounding its response in the selected source. The project addresses this by combining semantic retrieval with language generation.

2.2 Aim

To design and document an end-to-end web chatbot that retrieves relevant medical text and uses a generative language model to provide a concise, context-aware response.

2.3 Objectives

The key objectives are to load a medical PDF, divide its content into manageable chunks, represent those chunks as embeddings, store and retrieve vectors, connect retrieval to an LLM through LangChain, provide a simple browser interface, and encourage a medical-only response policy.

2.4 Expected Outcome

The expected outcome is a working prototype that demonstrates the full question-to-answer pipeline. Response quality depends on the coverage and quality of the indexed document, retrieval relevance, model behaviour, and the configuration of external services.

2.5 Project Deliverables

The main deliverables are the web application, reusable helper functions for document processing, a prompt definition, an indexing script, a chat template, a stylesheet, and a configuration guide. Together these files describe both the preparation stage and the live question-answering stage. A successful academic demonstration should also include setup instructions, test evidence, screenshots, and a clear statement of what the prototype can and cannot do.

CHAPTER 3 – BACKGROUND AND RELATED CONCEPTS

3.1 Generative AI

Generative Artificial Intelligence creates new content based on patterns learned during training. A language model can produce natural-language responses, summarize text, and follow instructions. However, fluent language is not proof of correctness, especially in health-related settings.

3.2 Medical Chatbots

Medical information chatbots can help users navigate educational material, understand terminology, and locate information. They should not independently diagnose conditions or prescribe treatment. A responsible implementation makes its scope clear and directs users to qualified clinicians for individual medical decisions.

3.3 Retrieval-Augmented Generation

RAG combines information retrieval with text generation. The retrieval component searches a knowledge collection for passages related to a question. The generation component uses the question and retrieved passages to produce a response. This supports traceability to a defined knowledge source but does not guarantee that every answer is correct.

3.4 Semantic Search

Semantic search compares vector representations of text to estimate meaning similarity. It can retrieve passages that use different wording from the user's question. Retrieval performance depends on the embedding model, chunk design, index settings, and query formulation.

3.5 Language Models and Context

A language model generates a sequence of words based on the input and the patterns it learned during training. Providing retrieved passages gives the model a local source of information to consider while answering. The model can still misread the context or combine it with unsupported assumptions, so the final answer should be checked against the evidence rather than judged only by fluency.

CHAPTER 4 – EXISTING AND PROPOSED SYSTEM

4.1 Existing Approach

A user may search a medical PDF manually, use ordinary keyword search, or ask a general chatbot. Manual search takes time; keyword search may depend on exact terms; and a general model may produce content that is not supported by a chosen reference.

4.2 Proposed Approach

CareBot accepts a question through a web interface, invokes a retrieval chain, searches a Pinecone index, and passes retrieved context to a Cohere language model. The answer is returned to the browser as a JSON response. The supplied prompt restricts the assistant to medical and healthcare topics and asks it to keep answers short.

4.3 Comparison

The proposed system adds semantic retrieval and a document-grounding step. It remains dependent on external services and the quality of its source material. It should be viewed as a prototype rather than a replacement for professional medical care.

Aspect	Common Basic Approach	CareBot Design
Search	Manual / keyword search	Semantic vector retrieval
Response	User reads source directly	LLM composes concise answer
Grounding	Depends on user search	Retrieved passages supplied as context
Limit	Time-consuming / exact words	Depends on source and retrieval quality

4.4 Advantages of the Proposed Design

The proposed design offers a conversational entry point, semantic matching, and a consistent prompt for generated answers. It can reduce the effort required to navigate a lengthy reference document and provides a useful demonstration of modern AI integration. Its modular structure also allows the retrieval model, vector database, prompt, and frontend to be improved independently.The project also demonstrates that successful AI development involves more than generating fluent text.

4.5 Trade-offs

The system introduces dependencies that a manual PDF search does not have: network connectivity, service credentials, vector-index availability, model latency, and usage costs. The generated answer may also omit details present in the retrieved source. Therefore, the design is most suitable as an assisted information interface, with the original reference remaining available for verification.

4.6 System Flexibility

The proposed CareBot system follows a modular architecture in which different components perform separate functions. The frontend manages user interaction, Flask handles incoming requests, the retrieval component searches the medical reference document, and the language model generates responses using the retrieved information. This separation makes the application easier to understand, maintain, and improve.

The modular structure allows developers to modify individual components without redesigning the complete application. For example, the prompt can be updated to improve response instructions, and retrieval settings can be adjusted to experiment with different search results. The frontend can also be redesigned to improve readability and user experience.

CHAPTER 5 – REQUIREMENT ANALYSIS

5.1 Functional Requirements

The system should display a chat page, accept a text question, invoke the retrieval chain, return a generated reply, and load its credentials from environment configuration. A separate indexing step prepares the medical PDF and stores its vectors in the configured index.

5.2 Non-Functional Requirements

The interface should be simple to use, responses should be reasonably timely, and the service should be maintainable. Reliability, confidentiality, accessibility, observability, and safe failure handling are important for healthcare-related applications.

5.3 Hardware and Software

A development machine capable of running Python and the embedding libraries is required. Internet access is needed for model downloads and hosted services. Software requirements include Python, Flask, LangChain-related packages, Hugging Face embedding support, Pinecone access, Cohere access, and environment-variable configuration.

Requirement Type	**Requirement**
Functional	Submit question and display reply
Functional	Retrieve top relevant text chunks
Functional	Generate answer from retrieved context
Non-functional	Protect API credentials
Non-functional	Handle service errors safely
Non-functional	Provide clear health disclaimer

5.4 User Requirements

A user should be able to identify the purpose of the chatbot, enter a clear question, understand when the system is processing, and read the answer without needing technical knowledge. The interface should also explain that responses may be incomplete and should not be treated as personal diagnosis or treatment. Error messages should describe the next practical step without exposing internal keys or stack traces.

5.5 Operational Requirements

The developer needs a repeatable installation process, a correctly configured vector index, valid provider credentials, and a documented method to launch the service. Dependencies should be versioned to reduce compatibility problems. Before each demonstration, the developer should verify the index, connectivity, environment variables, and basic query flow.

5.6Reliability and Maintainability

The application should handle common errors such as invalid requests, missing credentials, network failures, and unavailable vector indexes. Clear error messages can help users understand when the system cannot process a question. Developers should also maintain logs carefully without storing unnecessary sensitive information.

Regular maintenance is required to keep the application functional. This includes reviewing package compatibility, checking API configurations, updating the medical reference document when appropriate, and verifying the vector index. Testing after every major update can help identify errors before they affect users.

The modular structure supports maintenance by making individual components easier to inspect and modify. With proper documentation, secure configuration, and regular testing, the application can be maintained more effectively.

CHAPTER 6 – SYSTEM ARCHITECTURE

6.1 Architecture Overview

The application is divided into a presentation layer, Flask application layer, RAG orchestration layer, and external retrieval and generation services. The browser sends a message to the backend. The backend invokes the retrieval chain and returns the answer to the browser.

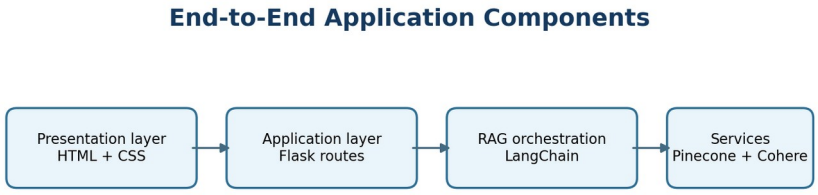
6.2 Main Components

The HTML template renders the chat interface and the CSS controls its appearance. Flask serves the page and handles the POST endpoint. Helper functions load and split PDF content and initialize embeddings. LangChain links the retriever and LLM. Pinecone stores vectors, while Cohere generates the final response.

6.3 Architecture Benefits

Separating interface, orchestration, indexing, retrieval, and generation makes the project easier to understand and modify. Each component can be tested separately. In production, additional layers for authentication, request validation, logging, rate limiting, and clinical governance would be needed.

Figure 6.1: System components and service boundaries



CHAPTER 7 – DATASET AND KNOWLEDGE PREPARATION

7.1 Source Document

The repository contains a medical reference PDF under the Data directory. The helper module uses a PDF loader to extract document text. The exact coverage and authority of this book should be reviewed before relying on the chatbot's answers.

7.2 Text Extraction

PDF loading converts pages into document objects that can be processed by the pipeline. Extraction quality can be affected by scanned pages, unusual layouts, tables, missing fonts, and OCR errors. Poorly extracted text may cause incorrect or incomplete retrieval.

7.3 Chunking

The helper module uses RecursiveCharacterTextSplitter with a chunk size of 500 and an overlap of 20. Chunking breaks long text into smaller pieces suitable for embedding and retrieval. Overlap helps preserve context around boundaries, while excessively small or large chunks may reduce retrieval usefulness.

7.4 Data Quality

Before indexing, the source should be checked for duplication, outdated guidance, missing pages, and inconsistent formatting. A future version should retain page metadata so responses can show where supporting passages originated.

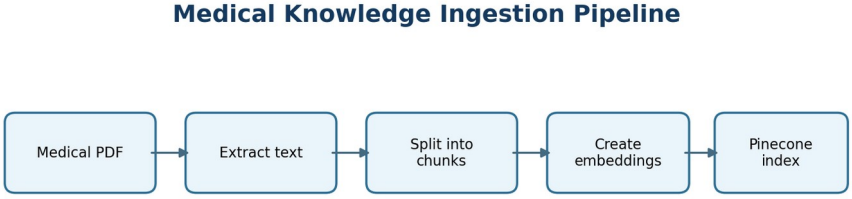


Figure 7.1: Knowledge ingestion pipeline

7.5 Chunk Design

Chunk size affects how much context is available for each retrieved result. Very small chunks may lose important surrounding explanations, while very large chunks may contain unrelated details and use more model context. Overlap can preserve continuity at boundaries, but it should be selected through evaluation rather than assumed to be optimal.

7.6 Source Governance

A medical knowledge collection should record its title, publisher, edition, publication date, and any applicable usage restrictions. When a source is replaced or updated, the index should be rebuilt and the application regression-tested. A future version should preserve page-level metadata so that users and reviewers can trace generated answers back to the source.

7.7 Knowledge Preparation Summary

Knowledge preparation is a fundamental part of the CareBot application because it determines what information can be retrieved during question answering. The process begins with loading the medical PDF and extracting its text. The text is then divided into smaller chunks using the configured text splitter.After chunking, the text segments are converted into embeddings and stored in the Pinecone vector database. During a user request, these stored representations support similarity-based retrieval, allowing the system to identify passages related to the submitted question.

Each stage contributes to the final quality of the response. Errors in text extraction can affect chunking, poor chunking can reduce retrieval relevance, and irrelevant retrieval can lead to incomplete generated answers. Therefore, the knowledge preparation process should be tested and maintained whenever the source document or processing configuration changes.

In conclusion, effective knowledge preparation provides the foundation for document-based question answering. By organising the medical reference document into searchable text segments, the CareBot project demonstrates how a large document can be transformed into a knowledge source for a conversational AI application.

CHAPTER 8 – EMBEDDINGS AND VECTOR SEARCH

8.1 Text Embeddings

An embedding model maps text into a numerical vector. Text with related meanings may be represented by nearby vectors, enabling similarity-based retrieval. The project initializes Hugging Face embeddings using sentence-transformers/all-MiniLM-L6-v2.

8.2 Vector Index

Pinecone is used as the vector database. The application connects to an existing index named carebot and creates a retriever that requests the three most similar results. Index creation and population are handled separately from normal chat requests.

8.3 Retrieval Settings

The code configures similarity search with k=3. This means up to three retrieved passages are supplied to the answer chain. The best value depends on the document, question type, context length, and evaluation results.

8.4 Practical Considerations

The embedding model used at query time must match the model used when creating the index. Index dimensions and metadata must also be consistent. Indexing should be repeatable, and changes to the source document should trigger a controlled re-indexing process.

8.5 Similarity Retrieval

The retriever compares the question representation with the vectors stored in the index and returns the closest candidates according to the configured search method. Similarity does not necessarily mean that a passage fully answers the question; it only indicates that the passage is close under the selected representation and metric. Retrieved passages therefore need to be inspected for relevance and sufficient evidence.

8.6 Index Maintenance

The index should be created with dimensions and configuration compatible with the selected embedding model. Re-indexing should be controlled so that old and new document versions are not unintentionally mixed.

CHAPTER 9 – RETRIEVAL-AUGMENTED GENERATION

9.1 RAG Pipeline

At runtime, the question is passed to a retrieval chain. The retriever searches the Pinecone index for semantically similar chunks. The selected chunks are combined with the question and supplied to the language model. The model produces an answer which the application sends back to the client.

9.2 LangChain Orchestration

The application uses LangChain utilities to create a document-combination chain and a retrieval chain. This orchestration reduces the need to manually implement each intermediate step and provides a clear path from retrieval to generation.

9.3 Grounding and Limits

Retrieved context can improve relevance, but the model may still misinterpret it, omit important caveats, or answer beyond the available evidence. A robust version should display source passages, evaluate answer faithfulness, and abstain when retrieval confidence is low.

9.4 Runtime Output

The Flask endpoint reads the message from JSON, invokes the RAG chain, and returns a JSON object containing the reply. Request validation and error handling should be added to manage empty inputs, service failures, and malformed requests.

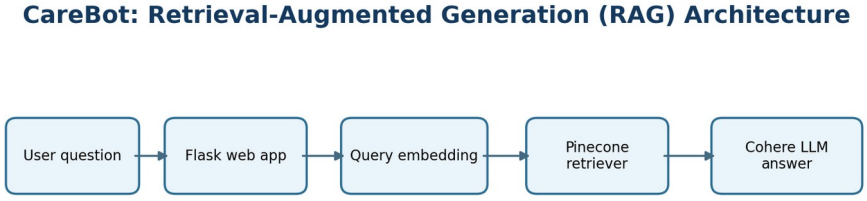


Figure 9.1: Question retrieval and answer generation

9.5 Context Construction

Context construction is an important stage in the Retrieval-Augmented Generation (RAG) process. It refers to the method of collecting, organising, and presenting the information retrieved from the knowledge source before sending it to the language model. The quality of the context has a direct influence on how effectively the model can understand the user's question and generate a relevant response.

In the CareBot project, when a user submits a medical question, the retrieval component searches the Pinecone vector database and identifies relevant text chunks from the medical reference document. These retrieved passages are then supplied to the language model through the configured LangChain workflow. The model uses the question and the retrieved context to generate a concise answer.

The retrieved passages should contain sufficient information to explain the topic being asked about. If the context contains only a small portion of the required explanation, the model may produce an incomplete answer. Similarly, if unrelated passages are included, the model may become distracted by information that does not directly address the question. Therefore, selecting and organising the retrieved information is an important part of improving answer quality.

9.6 Evidence and Traceability

A useful enhancement is to display the sources that contributed to the answer. Source titles and page numbers let a user inspect the original information and identify when the answer is incomplete. Source citations can also improve the transparency of the application. Instead of presenting an answer without explanation, the chatbot could identify the document passages that support its response. Users would then have a clearer understanding of the information used to prepare the answer. This is especially valuable when a response contains technical terminology or a simplified explanation of a complex medical concept.

CHAPTER 10 – PROMPT DESIGN AND RESPONSE POLICY

10.1 System Prompt

The supplied prompt identifies the assistant as CareBot and restricts it to medical and health-related questions. It instructs the model to politely decline out-of-scope questions, use retrieved context, admit when it does not know, and limit answers to a maximum of three sentences.

10.2 Why Prompting Matters

A system prompt sets expectations for response scope, tone, and format. It can reduce unwanted responses but cannot guarantee safety or correctness. Prompt rules should be combined with validation, monitoring, retrieval quality checks, and clinical review.

10.3 Recommended Response Behaviour

Answers should distinguish general information from personalized medical advice. The system should avoid claiming certainty when the source is incomplete, encourage professional assessment for symptoms or treatment decisions, and clearly direct emergencies to local emergency services. These are recommended improvements, not verified existing features.

10.4 Prompt Injection Risk

Retrieved documents and user questions may contain instructions that conflict with system rules. A hardened implementation should treat retrieved text as untrusted data, prevent it from overriding system instructions, and test adversarial prompts.

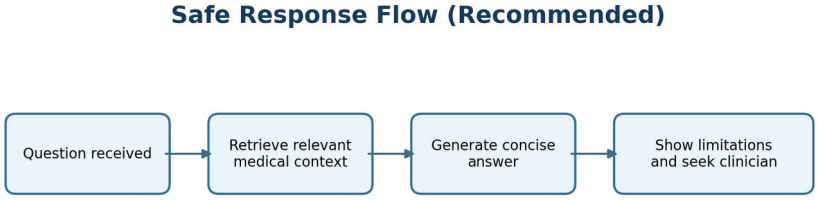


Figure 10.1: Recommended safe response flow

10.5 Response Format

A consistent response format is an important part of the CareBot system because it helps users understand the information provided by the chatbot. Medical information should be presented in a clear, simple, and organized manner so that users can easily understand the response without confusion. The chatbot should avoid unnecessarily complicated medical terminology and provide explanations in simple English wherever possible.

The response format should be designed according to the type of question asked by the user and the information available in the retrieved medical document. For a simple medical question, CareBot can provide a short and direct explanation. For a question that requires more detail, the chatbot can organize the answer into meaningful paragraphs or bullet points. This approach improves readability and helps users identify the most important information.

10.6 Policy Validation

Prompt behaviour should be tested with medical questions, non-medical questions, ambiguous requests, requests for a diagnosis, and questions that have no supporting passage. Tests should also include attempts to override the instructions. These tests help reveal weaknesses, but they do not replace technical safeguards or professional safety review.

CareBot should also be tested with questions that are unrelated to healthcare. For example, a user may ask about general knowledge, entertainment, or unrelated technical topics. If the system is intended to answer questions only from its medical knowledge source, it should avoid providing unrelated information as though it were part of the medical reference document. The expected behaviour should be defined in the prompt. Depending on the project requirements, the chatbot may politely explain that the question is outside its intended scope or ask the user to provide a relevant medical question. This test helps confirm whether the chatbot follows its intended purpose.

CHAPTER 11 – TECHNOLOGY STACK

11.1 Python and Flask

Python is used for backend logic. Flask exposes a route for the chat page and a POST endpoint that receives questions and returns JSON. Its lightweight structure suits a small prototype.

11.2 LangChain

LangChain connects the retriever, prompt, and language model. The code uses a retrieval chain and a document-combination chain to pass retrieved context into the model. Package versions should be pinned and checked for compatibility.

11.3 Embedding and Storage Services

Hugging Face sentence-transformers provides embeddings, and Pinecone stores and retrieves vector representations. Cohere is used as the language model provider in the application. These hosted components require valid credentials and network access.

11.4 Frontend Technologies

The project includes a chat HTML template and a CSS stylesheet. The screenshots supplied in the repository demonstrate the intended user interface. Interface accessibility, mobile responsiveness, and clear safety messaging should be included in further iterations.

Component	**Role in Project**
Python / Flask	Backend routes and JSON response
LangChain	RAG chain orchestration
Hugging Face	Text embedding model
Pinecone	Vector storage and similarity retrieval
Cohere	Generative answer model
HTML / CSS	Browser-based chat interface

11.5 Integration Considerations

Each dependency has a distinct role and its own versioning requirements. The embedding package, LangChain integrations, Pinecone client, and Cohere client should be installed in compatible versions. A clean environment build is a useful check that the README and requirements file describe the project accurately.

The CareBot project uses Flask for backend development, LangChain to coordinate the Retrieval-Augmented Generation (RAG) workflow, Pinecone for vector storage and similarity search, and Cohere for language generation. The sentence-transformers/all-MiniLM-L6-v2 model is used to convert text into numerical embeddings. These embeddings help the application identify medical text passages that are semantically related to a user's question.

11.6 Rationale for the Stack

The chosen stack supports a relatively small prototype without requiring the developer to build vector search or language generation from scratch. Flask provides a straightforward web layer, LangChain coordinates the RAG flow, and hosted services provide vector storage and generation. This convenience also creates a dependency on provider availability, service policies, and network access.

The technology stack selected for CareBot is intended to support the development of a relatively small medical information chatbot without requiring the developer to implement every technical component from the beginning. Each technology provides a specific function, and the combination supports the main requirements of the Retrieval-Augmented Generation (RAG) architecture.

The stack includes Flask for the web application, LangChain for coordinating the RAG workflow, Pinecone for vector storage and similarity search, Cohere for language generation, and a sentence-transformer model for converting text into embeddings.

CHAPTER 12 – BACKEND IMPLEMENTATION

12.1 Application Initialization

The main application imports Flask utilities, the embedding helper, Pinecone vector-store integration, Cohere LLM support, LangChain chain builders, prompt definitions, and dotenv. It initializes the Flask application and loads environment variables.

12.2 Retrieval Setup

The code initializes embeddings and connects to an existing Pinecone index named carebot. A similarity retriever is configured to return three results. The chain is then assembled using the system prompt and the Cohere model.

12.3 Request Handling

The home route renders the chat template. The /get route accepts POST requests, reads the message from the JSON body, invokes the retrieval chain, and returns the generated answer in JSON. The supplied code logs the input and answer to the console.

12.4 Improvements

Before deployment, validate request shape and message length, handle missing credentials, catch provider timeouts, avoid logging sensitive user content, and disable Flask debug mode. Add authentication and request limits if access is not intended to be public.

12.5 Request Validation

A robust endpoint should confirm that the incoming request is JSON, that the message field exists, and that its value is a non-empty string within a reasonable length. Invalid requests should receive a predictable response code and a helpful message. Validation reduces accidental failures and helps protect the model endpoint from unnecessary or abusive requests.

12.6 Error Handling and Logging

The backend should distinguish configuration errors, retrieval failures, provider timeouts, and unexpected internal errors. User-facing responses should remain simple, while diagnostic logs should avoid secrets .

CHAPTER 13 – FRONTEND AND USER INTERACTION

13.1 Chat Interface

The chat interface is an important component of the CareBot application because it provides a simple and convenient way for users to communicate with the system. The interface is developed using HTML and CSS and is displayed through a web browser. It allows users to enter their questions, submit them to the application, and view the responses generated by the backend system. The main objective of the chat interface is to provide an easy-to-use environment where users can interact with the healthcare chatbot without requiring advanced technical knowledge.

The HTML template defines the basic structure of the chat page, including the message display area, text input field, send button, and other interface elements. CSS is used to improve the appearance of the page by controlling colours, spacing, fonts, alignment, and overall layout. These styling features help create a clear and organized interface that supports a smooth user experience.

13.2 Interaction Sequence

The interaction sequence describes how a question moves from the user interface to the backend and how the generated answer is returned to the user. CareBot follows a request-and-response approach in which the frontend collects the user's question and communicates with the server through the configured backend endpoint.

The process begins when the user opens the CareBot application in a web browser. The chat page displays the available input controls, allowing the user to enter a question. After entering the question, the user selects the send button to submit the message.

13.3 Interface Screenshots

The following images are taken from the supplied repository and illustrate the project interface. They are included as project artefacts rather than independent proof of clinical accuracy.

13.4 Accessibility and Responsiveness

A clear chat page should remain usable on desktop and mobile screens. Text contrast, readable font sizes, keyboard navigation, focus indicators, and descriptive error messages all contribute to accessibility. The send action should be clearly identified, and the interface should prevent accidental duplicate submissions while a request is still running.

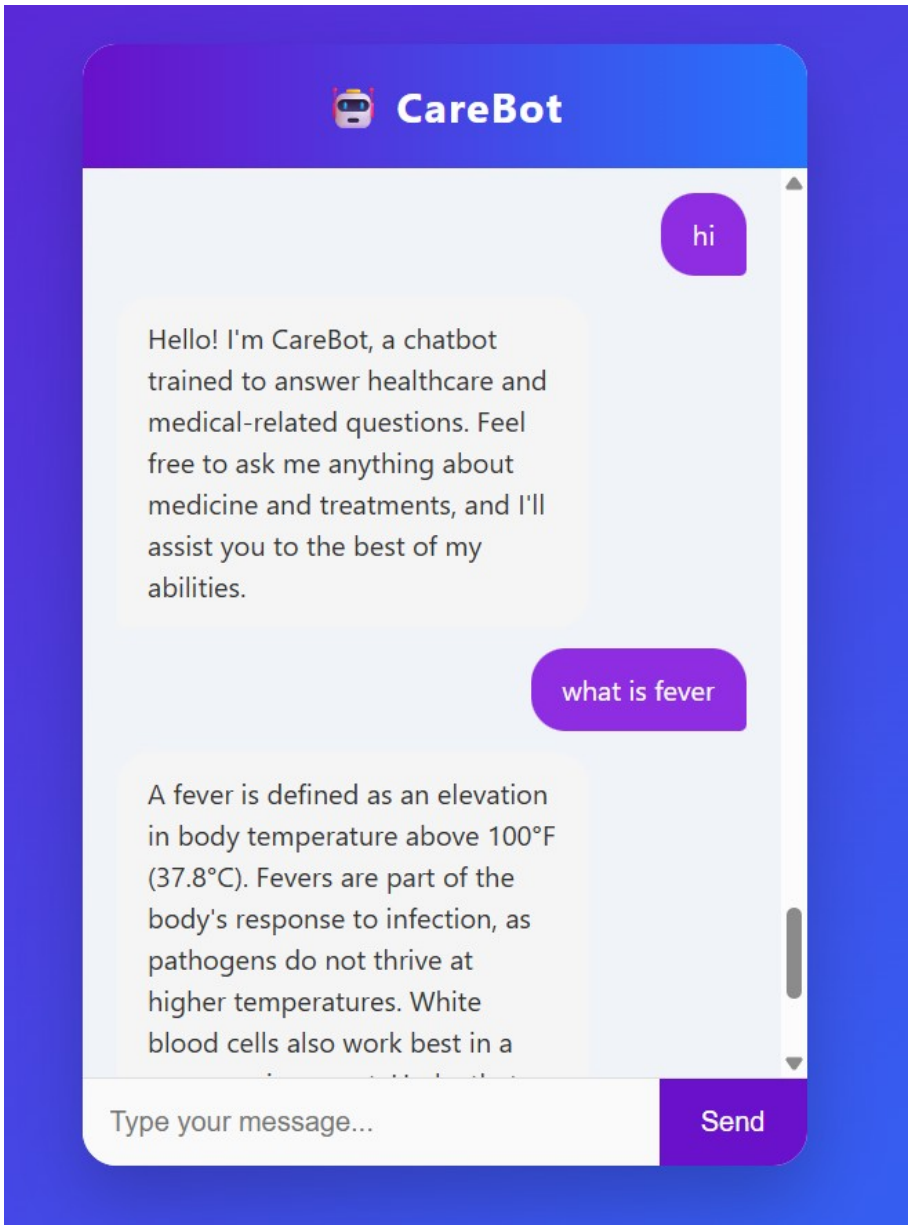


Figure 13.1: Supplied CareBot interface screenshot

CHAPTER 14 – END-TO-END WORKFLOW

14.1 Offline Indexing

The knowledge preparation stage loads the PDF, splits extracted text into chunks, computes embeddings, and stores vectors in the Pinecone index. This stage must be completed before queries can retrieve useful passages from the index.

14.2 Online Question Answering

The user enters a medical question. The backend calls the retrieval chain, which queries the vector store. Retrieved text is combined with the question and passed to Cohere. The generated answer is returned to the frontend.

14.3 Failure Paths

If credentials are absent, the index is unavailable, or a provider times out, the request may fail. The application should show a clear, non-technical message and log diagnostic information without exposing secrets or private health details.

14.4 Workflow Summary

The overall design links document preparation, semantic retrieval, prompt-guided generation, and a web interface. End-to-end testing should cover each stage as well as their integration.

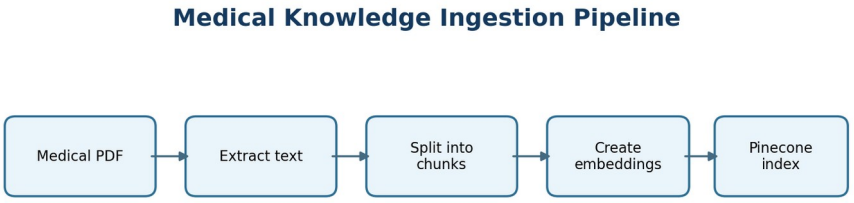


Figure 14.1: End-to-end data preparation and query workflow

14.5 Workflow Dependencies

The indexing workflow and online query workflow are related but separate. Indexing prepares the knowledge collection, whereas the web application assumes that a usable index already exists.

CHAPTER 15 – SETUP AND DEPLOYMENT

15.1 Environment Setup

Create a dedicated Python environment and install the packages listed in requirements.txt. The repository README describes environment variables for Pinecone and Cohere. Use private local configuration and never commit real API keys to source control.

15.2 Environment Variables

The project expects PINECONE_API_KEY and COHERE_API_KEY. A .env file can be used locally, while a deployed service should use its platform's secret-management feature. Rotate any credential that has been accidentally exposed.

15.3 Index Preparation

The vector index named carebot must exist and contain embeddings generated from the same embedding model. The repository includes store_index.py for indexing. Verify the data path, index dimensions, and successful completion of the ingestion process before starting the app.

15.4 Running the App

The README indicates that the application is started with python app.py and is served on port 8080. For production, use a production-grade WSGI server, HTTPS, restricted network access, health checks, and non-debug settings.

15.5 Deployment Readiness

Local execution is not equivalent to a secure public deployment. Before release, configure HTTPS, a production server, secret management, request limits, restricted access where appropriate, and health monitoring. Debug output should be disabled, and the service should return generic error messages instead of exposing internal implementation details.

15.6 Reproducibility

Keep setup commands, environment-variable names, index requirements, and model identifiers documented. A new developer should be able to create an isolated environment and reproduce the application .

CHAPTER 16 – TESTING STRATEGY

16.1 Purpose of Testing

Testing checks whether the application accepts valid questions, retrieves relevant information, generates an understandable response, and handles failures safely. Since this project is medical-facing, tests should also check out-of-domain requests and unsupported-answer behaviour.

16.2 Testing Levels

Unit tests can verify text splitting and request validation. Integration tests can check the retriever and LLM chain with controlled inputs. End-to-end tests can submit a question through the UI. Security tests should examine secret handling, input limits, prompt injection, and accidental logging.

16.3 Evaluation Dimensions

Useful evaluation dimensions include retrieval relevance, factual consistency with the source, answer completeness, readability, latency, availability, and safe abstention. A qualified medical reviewer should assess clinical correctness before any real-world use.

16.4 Test Environment

Tests involving hosted APIs require valid credentials and may incur usage charges. Use synthetic, non-identifying questions during development and do not submit real patient information unless the system has been approved for that purpose.

16.5 Regression Testing

A fixed set of representative questions should be run whenever the source document, embedding model, prompt, retrieval settings, or LLM changes. Comparing results over time can reveal whether a change improves relevance or introduces new errors. Store test cases and review outcomes in a controlled location without real patient identifiers.

CHAPTER 17 – TEST CASES AND OBSERVATIONS

17.1 Sample Test Cases

The following table provides a proposed test plan. Results must be filled in after running the tests in the target environment; no test execution results are claimed by this report.

Test ID	Scenario	Expected Result	Actual Result
TC-01	Valid medical question	Relevant concise response	To be tested
TC-02	Question outside healthcare	Polite scope refusal	To be tested
TC-03	Empty message	Validation message; no chain call	To be tested
TC-04	Missing API key	Safe configuration error	To be tested
TC-05	Vector index unavailable	Graceful service error	To be tested
TC-06	Question not in source	Admit insufficient evidence	To be tested
TC-07	Long input	Reject or safely limit input	To be tested
TC-08	Mobile viewport	Readable responsive interface	To be tested

CHAPTER 18 – CONCLUSION

The End-to-End Medical Care Generative AI project demonstrates how a document-grounded chatbot can be assembled from a web application, text-processing utilities, embeddings, a vector store, retrieval orchestration, and a generative language model. The supplied CareBot repository uses Flask for the web endpoint, Hugging Face sentence-transformer embeddings, Pinecone for similarity retrieval, LangChain for orchestration, and Cohere for answer generation.

The report has described the architecture, data preparation, retrieval and generation flow, interface, setup requirements, test strategy, and the main safety considerations. The design illustrates an accessible application of RAG to medical-document question answering. However, the repository does not by itself establish clinical correctness or production readiness.

The most important next steps are to test the application systematically, include source citations, improve error handling and privacy protections, and evaluate responses with qualified medical reviewers. CareBot should remain an educational information assistant and must not replace professional medical advice or emergency care.

The project provides a practical example of Retrieval-Augmented Generation applied to a medical reference document. It demonstrates how a web interface, embeddings, vector search, prompt instructions, and an LLM can be combined into a single user-facing workflow. This is a useful foundation for further learning and controlled experimentation with knowledge-grounded AI systems.

Further work should focus on measurable answer quality, source traceability, safe refusal, privacy protection, and reliable deployment. Before any clinical or public-facing use, the system would require formal security and privacy review, careful evaluation, and qualified medical oversight. Until then, it should remain an educational prototype and not be relied upon for diagnosis or treatment.

REFERENCES

1. Project repository: End-to-End Care Generative AI / CareBot, README.md and source files supplied for this report.

2. LangChain Documentation. Retrieval chains and document-combination concepts. https://python.langchain.com/docs/

3. Hugging Face. Sentence Transformers documentation. https://www.sbert.net/

4. Pinecone Documentation. Vector database and semantic search concepts. https://docs.pinecone.io/

5. Cohere Documentation. Language model API documentation. https://docs.cohere.com/

6. Flask Documentation. https://flask.palletsprojects.com/

7. Python Software Foundation. Python documentation. https://docs.python.org/3/

8. World Health Organization. Ethics and governance of artificial intelligence for health. https://www.who.int/publications/i/item/9789240029200

Accessed for report preparation: 13 October 2026. Verify the current URLs and institution-required citation style before final submission.